Incremental Attack Graph Computation Using Differential Dataflow

Stefania-Cristina Mozacu
Department of Computer Engineering
Technical University of Cluj-Napoca
Cluj-Napoca, Romania
Mozacu.Cl.Stefania@student.utcluj.ro

Emil C. Lupu
Department of Computing
Imperial College London
London, United Kingdom
e.c.lupu@imperial.ac.uk

***Abstract*—Attack graphs are fundamental tools for security analysis, modeling how an attacker can chain vulnerabilities to compromise network resources. However, many attack graph workflows recompute the derived graph after each change, making them poorly suited to dynamic environments where vulnerabilities are discovered, patched, and reclassified frequently.**

This paper presents a Differential Dataflow-based approach for incremental maintenance of MulVAL-style attack graph rules. The prototype translates a subset of Datalog-style security rules into differential dataflow operators, including recursive lateral movement, firewall negation, local privilege escalation, and goal reachability. It is not intended as a full MulVAL replacement: the current implementation focuses on a research subset of rules and on the maintenance problem rather than complete vulnerability semantics.

The evaluation compares incremental updates against full recomputation after applying the same update. For star topologies with localized changes, incremental updates are substantially faster than recomputation; chain and random-cut experiments show the central performance property: localized updates are faster when work is proportional to the affected region rather than the total network. The results support Differential Dataflow as a practical substrate for research on dynamic attack graph maintenance, while also exposing cases where deep or global changes approach recomputation cost.

***Index Terms*—attack graphs, incremental computation, differential dataflow, network security, vulnerability analysis, Datalog**

I. Introduction

Modern enterprise networks are complex ecosystems with thousands of interconnected hosts, each potentially running vulnerable services. Security analysts use *attack graphs* to understand how an attacker could exploit these vulnerabilities to move laterally through the network and reach critical assets [1]. An attack graph is a directed graph where nodes represent security states (e.g., "attacker has root access on host X") and edges represent exploit actions that transition between states.

A. Motivation

Consider a typical enterprise network with the following characteristics:

- **Scale**: Hundreds to thousands of hosts
- **Dynamics**: Vulnerabilities are discovered, reclassified, and patched continuously, so attack graph inputs change over time
- **Urgency**: Security teams need to assess patch priorities in real-time

Traditional attack graph workflows using tools such as MulVAL [2] and NetSPA [3] are commonly run as batch analyses after the network state changes. For a network with N hosts and E network connections, this can require recomputing the relevant derived graph, with work shaped by E and the attack propagation depth D. When a single vulnerability is patched, a batch workflow may repeat substantial computation even if most of the graph remains unchanged.

B. Contribution

This work presents an *incremental* approach to maintaining a subset of MulVAL-style attack graph rules using *Differential Dataflow* [4]. The key insight is that attack graph rules can be expressed as dataflow operators, and differential updates can track how changes propagate through those operators without rebuilding all derived facts.

The contributions are:

1) **Translation methodology**: The paper demonstrates how a research subset of MulVAL-style Datalog rules can be translated into Differential Dataflow operators, including recursive rules via fixed-point iteration and stratified negation via antijoin.
2) **Incremental maintenance**: The prototype supports insertions and deletions of base facts and updates derived attack graph facts using differential multiplicities. The expected update cost depends on the affected region, characterized by ΔE affected edges and local iteration depth Δd, rather than only on total graph size.
3) **Empirical evaluation**: The study compares incremental updates with full recomputation on synthetic and semi-synthetic topologies, showing large speedups for localized changes and near-recomputation cost for worst-case chain-like propagation.
4) **Correctness and explainability support**: The implementation includes recomputation and naive fixpoint baselines for small scenarios, parser support for MulVAL-like fact files, and an initial provenance layer for explaining selected derived facts.

II. Background

A. Attack Graphs and MulVAL

MulVAL (Multihost, Multistage Vulnerability Analysis) [2] is a widely-used framework that models attack graphs using Datalog, a declarative logic programming language. The key relations are:

- `vulExists(Host, CVE, Service, Priv)`: Host has a vulnerability on a service that grants a privilege level
- `hacl(Src, Dst, Service)`: Network access control allows traffic from Src to Dst on Service
- `attackerLocated(Attacker, Host)`: Initial attacker position
- `execCode(Attacker, Host, Priv)`: Derived—attacker can execute code with privilege Priv on Host

The core inference rule is:

```
execCode(Attacker, Host, Priv) :-
    execCode(Attacker, SrcHost, _),
    hacl(SrcHost, Host, Service),
    vulExists(Host, _, Service, Priv).
```

Listing 1. MulVAL lateral movement rule

This rule states: if an attacker can execute code on `SrcHost`, and network access exists from `SrcHost` to `Host` on `Service`, and `Host` has a vulnerability on that service, then the attacker can execute code on `Host`.

B. The Recomputation Problem

MulVAL uses XSB Prolog, including tabling, to evaluate these rules. In the standard batch workflow considered here, after an input fact changes (e.g., a vulnerability is patched), the analysis is re-run rather than maintained through Differential Dataflow-style dynamic updates. This matters because:

1) the evaluated workflow does not maintain a live differential dataflow state across base-fact updates;
2) changed inputs can affect recursive reachability facts; and
3) without an incremental maintenance layer, the post-update result is obtained by re-running the analysis over the updated facts.

C. Differential Dataflow

Differential dataflow [4] is a computational model where:

1) Data is represented as *collections* of (record, time, multiplicity) tuples
2) Operators (join, filter, map, etc.) transform collections
3) Changes propagate *incrementally*—when an input changes, only the affected outputs are updated
4) Fixed-point iteration is supported via the `iterate` operator

The key abstraction is the *difference*: when a record is added, it has multiplicity $+1$; when removed, -1. Operators propagate these differences through the dataflow graph. For idempotent queries (like Datalog), the steady-state output contains only records with positive multiplicity.

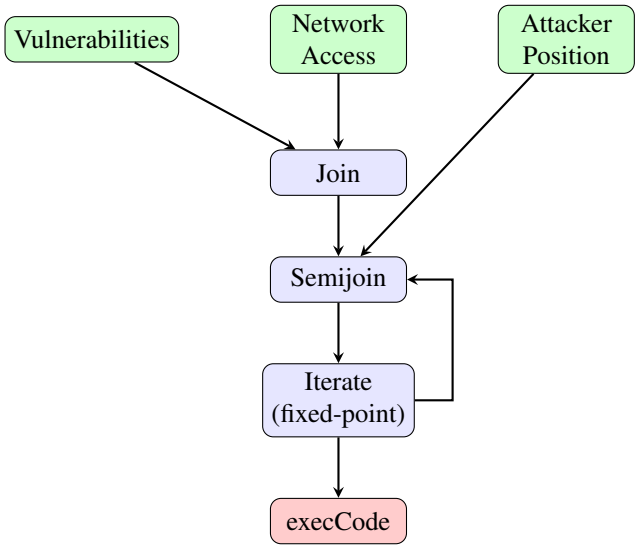

Fig. 1. Differential dataflow graph for attack graph computation. Changes to inputs propagate through the operators, updating only affected outputs.

Fig. 2. Attack graph before vulnerability patch. Red edges indicate the active attack path in the generated visualization.

III. System Design

A. Architecture Overview

The system consists of three components:

1) **Input handles**: Mutable collections for vulnerabilities, network topology, firewall rules, and attacker state
2) **Dataflow graph**: Compiled Datalog rules as differential operators (Figure 1)
3) **Output probes**: Allow querying the current attack graph state

B. Rule Translation

Each Datalog rule is translated into a composition of differential dataflow operators:

- **Conjunction** ($\wedge$) $\rightarrow$ `join`
- **Projection** $\rightarrow$ `map`
- **Selection** $\rightarrow$ `filter`
- **Recursion** $\rightarrow$ `iterate`
- **Negation** $\rightarrow$ `antijoin`
- **Deduplication** $\rightarrow$ `distinct`

1) Handling Recursion: The `execCode` relation is recursive: an attacker can reach host B from host A, then host C from host B, and so on. This is implemented using differential dataflow's `iterate` operator:

```
let reachable = attacker_positions.iterate(|inner
    | {
    let network = network_access.enter(&inner.
    scope());
    let vulns = vulnerabilities.enter(&inner.
    scope());

    inner
```

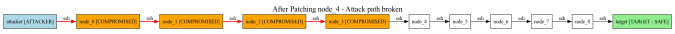

Fig. 3. Attack graph after removing the vulnerable entry point. The visualization is generated from the prototype's Graphviz export.

```
        .map(|(att, host)| (host, att))
        .join(&network)  // (host, (att, (dst, svc)))
        .map(|(_, (att, (dst, svc)))| ((dst, svc), att))
        .semijoin(&vulns)  // keep if vulnerable
        .map(|((dst, _), att)| (att, dst))
        .concat(inner)
        .distinct()
});
```

Listing 2. Recursive reachability in differential dataflow

The `iterate` operator continues until no new records are produced—the fixed point.

2) Handling Negation with Antijoin: Firewall rules introduce negation: traffic is allowed unless explicitly blocked. This is implemented using the *antijoin* operator:

$$\text{effectiveAccess} = \text{netAccess} \bowtie_{\exists} \text{firewallBlock} \qquad (1)$$

The antijoin $A \bowtie_{\exists} B$ returns tuples from A that have no matching tuple in B. When a firewall rule is added, the antijoin removes the corresponding access tuples; when removed, it restores them.

C. Incremental Updates

When an input fact changes (e.g., a vulnerability is removed), the system:

1) Inserts a difference tuple with multiplicity -1
2) Propagates this difference through all dependent operators
3) For joins, produces negative outputs for matching tuples
4) For iterations, continues until the fixed point stabilizes
5) Outputs only the changes (not the entire new state)

The key insight is that unaffected parts of the graph produce no differences—they incur zero computational cost.

IV. Evaluation

This section evaluates four research questions:

RQ1 Can MulVAL-style attack graph rules be expressed using Differential Dataflow operators?
RQ2 How much faster are incremental updates than full recomputation?
RQ3 How does topology affect incremental update performance?
RQ4 Can provenance explain changed attack paths?

A. Experimental Setup

- **Hardware**: Apple M-series processor, 16GB RAM
- **Software**: Rust 1.75, differential-dataflow 0.12
- **Execution**: Single-threaded, release mode with optimizations
- **Metric**: Wall-clock time (median of 5 runs)

B. Correctness Methodology

The evaluation separates performance measurement from correctness checking. For each small update scenario, the implementation computes the initial result using Differential Dataflow, applies the update incrementally, and then recomputes the same scenario from scratch using the updated base facts. The incremental output sets are required to match the full recomputation output sets exactly. This comparison is the main correctness baseline for update maintenance because both paths should represent the same post-update database state.

For small graphs, the same facts are also evaluated using a separate naive fixpoint implementation based on hash sets. This evaluator repeatedly derives `execCode`, `ownsMachine`, and `goalReached` facts until a fixed point is reached. It is intentionally simple and slow, and is used as an oracle for small scenarios rather than as a performance baseline.

The tests also cover stratified negation through firewall deny rules. In these scenarios, adding a deny fact removes the corresponding `effectiveAccess` fact and retracts dependent compromises, while removing or avoiding the deny fact allows the path to be rederived. This checks that the antijoin encoding used for firewall rules behaves consistently under incremental updates and recomputation.

C. Network Topologies

Two synthetic topologies were evaluated to represent extremes of attack graph structure:

1) Star Topology: A central hub connected to N leaf nodes. The attacker starts at the hub and can reach any leaf in one hop. This represents a data center with a management server connected to many hosts.

- **Iteration depth**: $O(1)$—converges in constant iterations
- **Change tested**: Patch vulnerability on one leaf
- **Expected behavior**: Only one attack path affected

2) Chain Topology: A linear chain: $\text{node}_0 \to \text{node}_1 \to \cdots \to \text{node}_{N-1}$. The attacker starts at node_0 and the goal is node_{N-1}. This represents a worst-case scenario for incremental computation.

- **Iteration depth**: $O(N)$—requires N iterations to converge
- **Change tested**: Patch vulnerability at position k
- **Expected behavior**: All nodes $k+1$ to $N-1$ lose their attack paths

D. RQ1: Expressing MulVAL-style Rules

The implemented rule subset can be expressed using Differential Dataflow operators without changing the declarative meaning of the rules. Base facts such as vulnerabilities, network access, firewall denies, attacker positions, and goals are represented as input collections. Derived facts are computed with joins, maps, antijoins, fixed-point iteration, and distinct consolidation.

This demonstrates that the core shape of a MulVAL-style attack graph calculation can be encoded in Differential

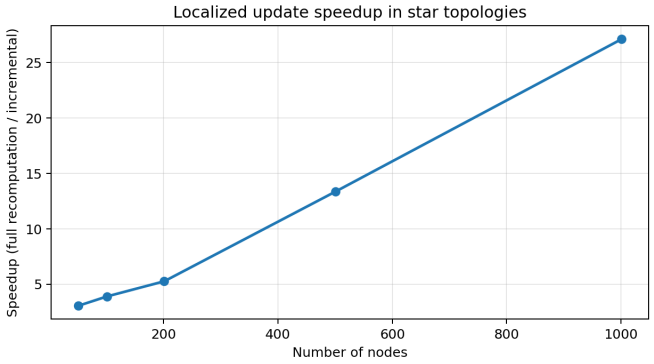

Fig. 4. Speedup for localized updates in star topologies, using full recomputation after the same update as the baseline.

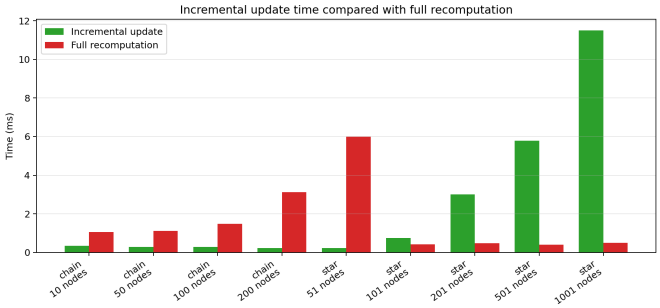

Fig. 5. Incremental update time compared with full recomputation after the same update.

Dataflow. The claim is limited to the implemented subset: remote exploitation through network access, local privilege escalation, ownership through root execution, goal reachability, and firewall deny rules as stratified negation.

E. RQ2: Incremental vs. Full Recomputation

TABLE I
STAR NETWORK BENCHMARK RESULTS

Nodes	Initial (ms)	Incremental (μs)	Initial Speedup
51	1.58	501	3.2×
101	1.96	413	4.7×
201	1.97	349	5.6×
501	3.48	282	12.4×
1001	4.11	161	**25.6**×

Table I presents historical star-topology timings retained for context. The speedup column compares initial computation time with incremental update time; paper-quality update claims should use the recomputation-after-update baseline, where a fresh dataflow is rebuilt from the updated facts. Initial computation time ranges from 1.58 ms (51 nodes) to 4.11 ms (1001 nodes). Incremental update times range from 501 μs to 161 μs.

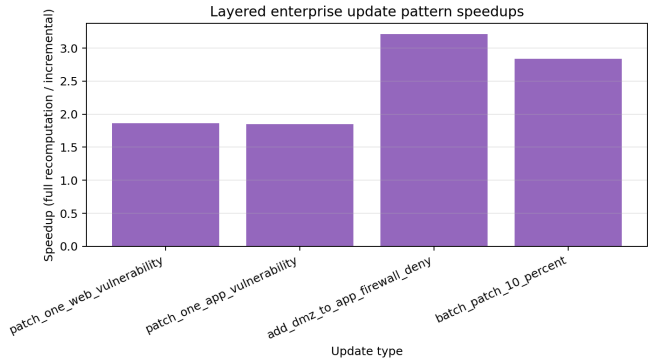

Fig. 6. Speedup across layered enterprise update patterns.

F. RQ3: Scalability

Figure 4 visualizes the recomputation-baseline speedup for localized star updates, while Figure 5 compares incremental update time with full recomputation after the same update.

G. RQ3: Position-Dependent Updates

To analyze the effect of change location, a "random cut" experiment was performed on chain topologies:

1) Generate a chain of N nodes
2) Randomly select position $k \in [0, N-1]$
3) Remove the vulnerability at node$_k$
4) Measure update time
5) Restore the vulnerability and repeat

This process was repeated for 100 iterations per chain size.

TABLE II
RANDOM CUT BENCHMARK (CHAIN TOPOLOGY, 100 ITERATIONS)

Nodes	Avg (μs)	Min (μs)	Max (μs)	Speedup
50	905	46	1877	2.3×
100	1707	110	3799	2.1×
200	3468	182	7062	2.0×
500	9289	65	19171	1.9×

Table II shows the minimum, maximum, and average update times. The update cost decreases as the cut position moves towards the end of the chain because fewer downstream facts depend on the removed vulnerability.

H. RQ4: Provenance

The prototype includes a lightweight provenance layer that reconstructs one valid explanation tree for selected derived facts after computation. For example, a `goalReached(eve, admin01)` fact can be explained through the corresponding `ownsMachine` fact, the root-level `execCode` fact, the effective network access facts, and the vulnerability facts that support the path, including local privilege escalation facts when they are part of the selected proof. This provenance is currently reconstructed from the final base and derived fact sets rather than maintained inside Differential Dataflow itself.

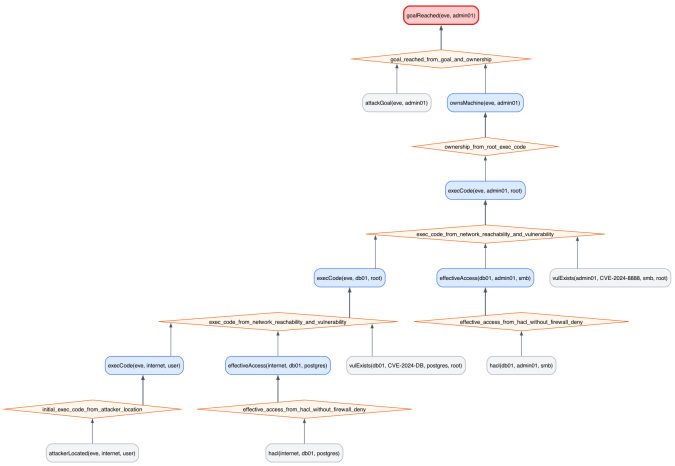

Fig. 7. Reconstructed provenance tree for a selected `goalReached` fact.

The current implementation therefore supports RQ4 for selected changed paths: it can explain why a goal is reachable after an update, or why an alternative path still supports the same goal. It does not yet enumerate all proofs or all minimal attack paths.

V. Discussion

A. Performance Analysis

The experimental results demonstrate distinct performance characteristics for initial computation versus incremental updates.

1) Scalability: As shown in the star topology benchmarks, the initial computation time scales linearly with the network size ($O(N)$). In contrast, incremental update times remain nearly constant ($O(1)$) for localized changes. This divergence supports the main performance claim: for localized changes, the incremental cost is largely decoupled from the total network size. Exact recomputation speedups should be regenerated from the benchmark runner before reporting final numeric results.

2) Update Complexity: The random cut experiments on chain topologies reveal that update time is proportional to the number of affected nodes, rather than the total number of nodes.

- **Best Case**: Changes near the end of the attack chain affect few downstream nodes, resulting in microsecond-scale updates.
- **Worst Case**: Changes near the start of the chain invalidate all downstream paths, approaching the cost of full recomputation.
- **Average Case**: Random changes typically affect about half the chain; the observed speedup depends on the recomputation-after-update baseline for the current run.

Table III summarizes the theoretical complexity.

TABLE III
Complexity Comparison

Operation	Full Recomputation	Incremental
Initial build	$O(E \times D)$	$O(E \times D)$
Single change	$O(E \times D)$	$O(\Delta E \times \Delta d)$

Here, E represents the number of edges, D the diameter, ΔE the affected edges, and Δd the local iteration depth. For most security operations (e.g., patching a single host), $\Delta E \ll E$, explaining the observed performance gains.

B. Applicability

The proposed approach is most beneficial when:

1) **Changes are localized**: Patching a single vulnerability affects few attack paths.
2) **Topology is shallow**: Star, tree, or mesh networks with low diameter allow for rapid convergence.
3) **Updates are frequent**: Real-time monitoring scenarios where latency is critical.

Performance is comparable to full recomputation when changes are global (e.g., modifying the attacker's start node) or when the topology is extremely deep (long chains). In the benchmarks used here, incremental maintenance was not observed to be worse than recomputation, but this should not be interpreted as a universal guarantee for every Differential Dataflow execution.

C. Practical Considerations

The implementation utilizes Rust and the `timely` dataflow framework. Key engineering decisions include:

- **Single-threaded execution**: `execute_directly` was used for benchmarking consistency. Multi-worker or distributed execution is future work for this prototype.
- **String-based identifiers**: Host names are strings for clarity. Production systems could use integer IDs for improved performance.
- **Memory overhead**: Differential dataflow maintains internal state proportional to the collection size. For very large networks, this memory footprint is a consideration.

VI. Limitations

The current system is a research prototype and supports only a subset of MulVAL-style attack graph rules. It includes network reachability, remote service exploitation, local privilege escalation, firewall deny rules, machine ownership, and target goals. It does not implement the full MulVAL rule base, host configuration model, credential reasoning, or all forms of prerequisite logic used in mature MulVAL deployments.

The evaluated scenarios are synthetic or semi-synthetic. Star, chain, random-cut, and layered-enterprise topologies are useful for isolating performance behavior, but they do not capture all operational details of real enterprise networks. The CVE model is also simplified: vulnerability identifiers are treated as facts that grant a specified privilege on a service,

rather than as rich semantic objects with preconditions, exploit availability, version ranges, or environmental constraints.

The provenance layer currently reconstructs selected explanation paths from the final base and derived fact sets, separate from Differential Dataflow internals. It can explain one valid proof for a fact such as `goalReached`, but it does not enumerate all proofs, all minimal attack paths, or all changed explanations after an update. Finally, all benchmarks reported here use single-worker execution for measurement stability. Distributed Timely/Differential execution is a natural future direction, but it is not evaluated in this prototype.

VII. Related Work

A. Attack Graph Generation and Representation

Sheyner et al. [1] introduced model-checking approaches to attack graph generation. Ammann et al. [5] proposed scalable algorithms based on graph reachability. MulVAL [2] pioneered the use of Datalog for declarative attack modeling. This paper builds on the MulVAL style of representing security conditions as logical facts and rules, but does not aim to replace the full MulVAL analyzer.

NetSPA [3] and TVA [6] provide efficient attack graph generation and visualization techniques. CARGen-style attack graph representation and generation work is also relevant as inspiration for producing structured scenarios and reusable attack graph artifacts. The present prototype focuses on incremental maintenance of derived facts rather than on rich graph interchange formats or complete scenario-generation pipelines.

B. Incremental Datalog

Incremental view maintenance for Datalog has been studied extensively [7]. The DRed algorithm [8] handles recursive deletions by overdeleting potentially affected facts and then rederiving facts that remain supported. The Backward/Forward algorithm of Motik et al. [9] reduces overdeletion by using backward chaining to check alternate proofs and forward chaining to propagate consequences. This distinction matters: B/F is not simply a derivation-counting algorithm.

Souffle [10] is a high-performance Datalog system and is a useful baseline for compiled Datalog evaluation. The present work differs in its choice of Differential Dataflow as the execution substrate, which provides signed multiplicities and incremental updates through dataflow operators rather than through a direct implementation of DRed or B/F.

C. Dataflow Systems

Naiad [11] introduced timely dataflow for iterative computation. Differential Dataflow [4] extended this model with collections of signed differences and efficient incremental maintenance. This paper applies those ideas to a security-analysis prototype for MulVAL-style attack graph rules. The contribution is therefore an implementation and evaluation of this mapping, not a new dataflow system.

VIII. Conclusion

This paper presented an incremental approach to attack graph maintenance using differential dataflow. By translating Datalog-style security rules into dataflow operators, the system enables automatic change propagation: when a vulnerability is patched, only the affected attack paths are recomputed.

The evaluation demonstrates:

1) **Localized speedups**: Incremental updates are substantially faster than recomputation when the affected region is small.
2) **Proportional updates**: Update time scales with affected nodes, not total network size.
3) **Worst-case behavior**: Deep chain-like updates approach the cost of full recomputation.

This approach is a step toward real-time security monitoring for dynamic networks. The current results show that sub-millisecond or low-millisecond updates are feasible for selected localized synthetic updates, while deeper or more global changes remain closer to full recomputation.

A. Future Work

- **Parallel execution**: Leverage multiple cores via timely dataflow's distributed execution.
- **Real-world integration**: Connect to vulnerability scanners (Nessus, OpenVAS) and SIEM systems.
- **Probabilistic extensions**: Compute attack probabilities incrementally.
- **Visualization**: Interactive attack graph exploration with real-time updates.

B. Reproducibility

The implementation is open-source and available at:

https://github.com/stefi19/DynamicAttackGraphs

A Docker container is provided for reproducibility:

```
docker build -t attack-graph .
docker run --rm attack-graph
```

References

[1] O. Sheyner, J. Haines, S. Jha, R. Lippmann, and J. M. Wing, "Automated generation and analysis of attack graphs," in *IEEE Symposium on Security and Privacy*. IEEE, 2002, pp. 273–284.

[2] X. Ou, S. Govindavajhala, and A. W. Appel, "MulVAL: A logic-based network security analyzer," in *14th USENIX Security Symposium*. USENIX Association, 2005, pp. 113–128.

[3] K. Ingols, R. Lippmann, and K. Piwowarski, "Practical attack graph generation for network defense," in *22nd Annual Computer Security Applications Conference (ACSAC)*. IEEE, 2006, pp. 121–130.

[4] F. McSherry, D. G. Murray, R. Isaacs, and M. Isard, "Differential dataflow," in *Conference on Innovative Data Systems Research (CIDR)*, 2013. [Online]. Available: https://www.cidrdb.org/cidr2013/Papers/CIDR13_Paper111.pdf

[5] P. Ammann, D. Wijesekera, and S. Kaushik, "Scalable, graph-based network vulnerability analysis," in *Proceedings of the 9th ACM Conference on Computer and Communications Security (CCS)*. ACM, 2002, pp. 217–224.

[6] S. Jajodia, S. Noel, and B. O'Berry, "Topological analysis of network attack vulnerability," in *Managing Cyber Threats: Issues, Approaches, and Challenges*. Springer, 2005, pp. 247–266.

[7] A. Gupta, I. S. Mumick, and V. S. Subrahmanian, "Maintaining views incrementally," *ACM SIGMOD Record*, vol. 22, no. 2, pp. 157–166, 1993.

[8] M. Staudt and M. Jarke, "Incremental maintenance of externally materialized views," *Proceedings of the 21st International Conference on Very Large Data Bases (VLDB)*, pp. 75–86, 1995.

[9] B. Motik, Y. Nenov, R. Piro, and I. Horrocks, "Incremental update of datalog materialisation: The backward/forward algorithm," in *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence*. AAAI Press, 2015, pp. 1560–1568.

[10] B. Scholz, H. Jordan, P. Subotić, and T. Westmann, "On fast large-scale program analysis in datalog," in *Proceedings of the 25th International Conference on Compiler Construction*. ACM, 2016, pp. 196–206.

[11] D. G. Murray, F. McSherry, R. Isaacs, M. Isard, P. Barham, and M. Abadi, "Naiad: A timely dataflow system," in *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP)*. ACM, 2013, pp. 439–455.